

P4-TURNet: Enabling NAT Traversal through P2P Relay Networks and Programmable Switches

Eduardo C. Paim, Alberto Schaeffer-Filho

Federal University of Rio Grande do Sul, Porto Alegre, Brazil

E-mail: {eduardo.paim, alberto}@inf.ufrgs.br

Abstract—The growing popularity of high throughput and low latency applications introduces new challenges in adapting the current network design practices to fit these requirements. With the advent of software-defined and programmable networks, there are new opportunities to optimize the operation of established mechanisms, such as peer-to-peer (P2P) communication. In this paper, we propose P4-TURNet, a new system that enables NAT traversal by managing multiple programmable switches that act as P2P relay servers. We discuss how P4-TURNet can be employed to provide massive communication at scale to achieve restrictive performance requirements. We evaluate the system in a simulated environment and find that it enables a TURN server to handle 400 times more simultaneous users compared to traditional on-server packet relaying.

I. INTRODUCTION

In the current state of the Internet, the vast majority of its largest applications and services are designed as client/server architectures. The client/server architecture has a number of advantages over P2P communication, such as a centralized control, easier monitoring and debugging, well-known security threats and easier access to the user's data. It also adapted well to the surge of smartphones in the last decade, as it does not require a public IP address from clients nor a fast upload speed from the network.

The latest years brought into the spotlight new network requirements, as emerging technologies such as IoT networks, high definition video calls, digital twins and remote surgeries impose strict latency and throughput requirements to the network [1], [2]. Future applications also point towards even greater network requirements, as communication with VR devices, holographic concerts and smart cities are rising in popularity in scientific and industrial conferences. This trend offers a new opportunity to revisit P2P architectures. A P2P design favors (1) low latency, due to requiring a shorter path to communicate directly two clients; and (2) high throughput, as P2P architectures offload the resource costs of dealing with user traffic to the clients.

In the context of P2P architectures, one of the main practical issues is the need for a relay server to deal with clients without public IP addresses. Those relay servers add substantial latency and throughput penalties to the solution. In this paper, we present P4-TURNet, a system that tackles the NAT traversal issue by exploring the capabilities of software-defined networking and uses a network of programmable switches acting as relay servers. We first describe the components of the system and how they integrate with each other. Then,

we discuss how this architecture would satisfy the emerging network requirements. We evaluate the system in an emulated environment and find that the system vastly improves over traditional TURN servers. In particular, P4-TURNet is capable of handling 400 times more simultaneous users by offloading the relaying of packets to a programmable switch and by extending the TURN server functionality to enable the management of user allocations across multiple switches.

The remaining of this paper is organized as follows: Section II presents related concepts to NAT traversal and software-defined networks. Section III details the proposed system design. Section IV evaluates our proof-of-concept in an emulated environment and compares it with the traditional NAT traversal technique. Section V discusses the related work and Section VI concludes the paper.

II. BACKGROUND

A. NAT traversal

Network Address Translators (NAT) allow organizations to share a limited amount of public IP addresses between a larger set of clients. It works by translating the private (not globally unique) addresses in the internal network into public (globally unique) addresses before packets are forwarded onto another network. A peer wishing to receive connections from other peers must find a way to circumvent NAT to expose an IP address to the P2P network. In situations in which it is possible to know or compute the public address (Figures 1a and 1b, Cone strategy always has fixed mapping), it is sufficient to employ usual NAT traversal techniques such as hole-punching and the UPnP protocol. However, in the case of symmetric carrier-grade NATs (CG-NATs, largely deployed in corporate and wireless networks), the traditional techniques fail.

Under a symmetric NAT (Fig 1c), peers cannot guess the mapping they will get, which is different for every destination address. For this reason, symmetric NATs are called *address-dependent*. A has no solution but to use a relay server which receives packets from A in a client/server connection and sends each packet to B on behalf of A. The most popular protocol that describes this solution is the TURN protocol [3], which is part of the Session Traversal Utilities for NAT (STUN) standard. Although the protocol is quite old, today's P2P VoIP solutions still use TURN servers to deal with clients' ill-behaved NATs [4].

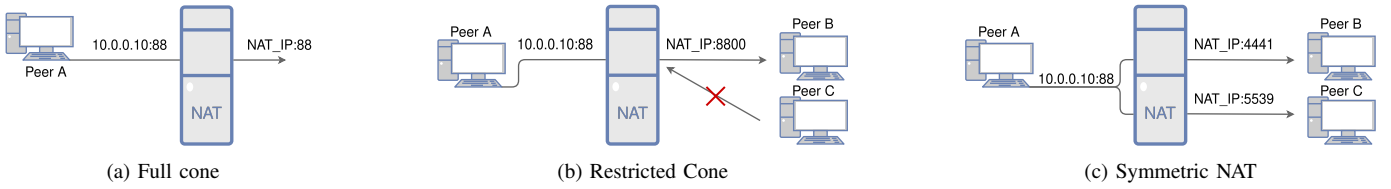


Fig. 1: Mapping strategies for NAT. The Full cone strategy is a fixed (and predictable) mapping address. Restricted Cone additionally requires the origin to initiate the connection first, before being allowed to receive packets.

B. Software-Defined Networks and Programmable Switches

Software-Defined Networking is a paradigm that separates the control-plane logic from the data-plane logic of a network device. This separation is done by means of a software controller that is capable of controlling the network traffic by adjusting the internal configuration of each switch (e.g. adding or removing entries from its match-action tables). A network administrator can interact with the control plane through an API to adjust the behaviour of the network.

Programmable switches are modern switch devices that allow the data plane logic to be customized by the network administrator using a language such as P4 [5]. This capability allows for the development and use of new network protocols or header logic without the need of subsequent vendor releases. Combined with SDN, a programmable switch network can implement the data plane logic necessary to implement custom network behaviour.

III. P4-TURNET DESIGN

A. Challenges

Server network I/O: A TURN server imposes a heavy network load on the host, as both the downlink and the uplink will be under constant load during the relaying session. Popular implementations of the TURN protocol note that, for VoIP connectivity, a TURN server can handle thousands simultaneous calls per CPU, compared to tens of thousands calls when only the STUN protocol is used (without relaying packets) [6]. In other words, relaying packets imposes a penalty of an order of magnitude on the number of simultaneous clients the server supports.

On the other hand, a programmable switch has a throughput as high as 12 Tbps. Recent studies [7] explored the use of a switch for relaying media content, and the results are promising. A TURN server offloading the relaying of packets to a programmable switch has the potential to overcome the resource constraints imposed by the hardware limitations of network interfaces, therefore supporting a much larger number of simultaneous clients on a single server.

Latency: The latency requirements for the emergent applications are shortening to values not achievable by usual client/server architectures. By having a network flow passing through a relay server means that each packet has to pass through many more network hops than it otherwise would have to if the peer could communicate directly to other peers. The current client-server solution to latency requirements is the

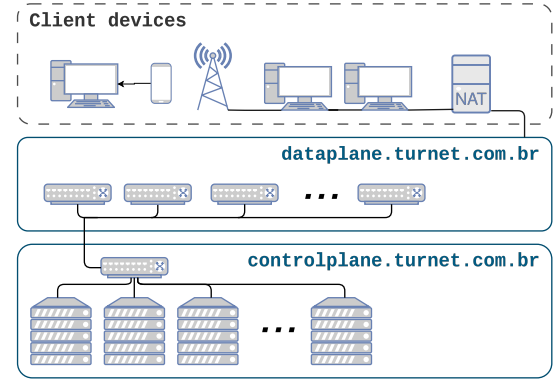


Fig. 2: P4-TURNET system overview.

adoption of Content Deliver Networks (CDNs), that became commonplace in the last decade in almost all major application services.

This work applies the same principle of CDNs, which is to bring the server close to the clients, by proposing a network of relay switches. Each programmable relay switch serves its closest peers to provide maximum throughput and latency benefits, essentially allowing any peer, even smartphone devices under the latest CG-NATs, to use P2P applications.

B. P4-TURNET System Overview

The system is composed of two main components: a logically centralized controller and a set of programmable switches, which are directly connected to the controller via a specific interface. The controller will manage the internal state of every switch, and will be responsible for creating and destroying connections on the entire system, which we will call the *presentation* and *termination* phases, respectively. The set of switches will be responsible for the relaying of packets, a *relaying* phase after the connection is already established. Figure 2 shows a high-level view of the system. The system is open to the Internet on two anycast domains (*dataplane* and *controlplane*). Clients will use the dataplane IP to allocate ports on the system that can be utilized to circumvent CG-NATs in P2P applications. The programmable switch's high-throughput capabilities ensure that the system can handle high traffic and user volume.

To simplify our explanation, in Section III-C we will first detail how the process works for a single switch and a single physical controller, and then expand it to a multiple switch architecture with multiple controllers in Section III-D.

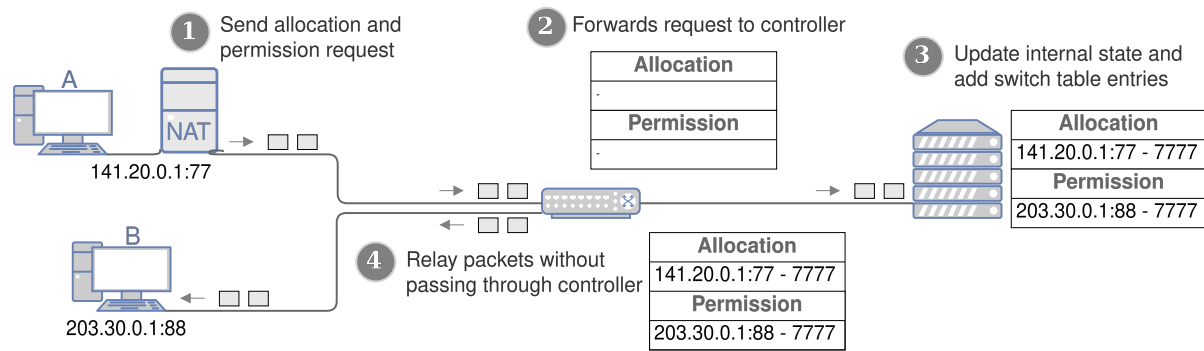


Fig. 3: Simplified system with two peers A and B, a programmable switch and a controller/TURN server

C. A simple architecture (one switch)

A peer A, under a CG-NAT with address-dependent mapping, is trying to connect to a peer B that is outside of a CG-NAT. As is common in NAT traversal protocols, peer A already knows the public address of peer B through a signaling protocol. Knowing that A cannot create a P2P connection with B without a fixed public address, it is necessary for A to “rent” an address on a relay server. This process is illustrated in Figure 3.

- 1) *Peer A sends an allocate and permission requests to the relay server:* An allocation request will inform the system that peer A is under a NAT and wants to “rent” a public facing IP and port address on the system. A permission request comes with an additional address (the IP:port of B) indicating that peer A wants to receive packets from that address.
- 2) *Switch forwards messages to control plane:* Upon receiving the requests from A, the switch will identify that A does not have an address rented in its internal state and will forward the requests to the server.
- 3) *Updating tables:* The server (controller) will create a mapping $(peerA_IP, peerA_Port) \rightarrow (relayIP, relayPort)$ on its internal state. It will also communicate with the switch and add this entry into the switch’s match-action table. Upon receiving the permissions request, The relay server will create a permission binding B’s IP and port to the relay IP and Port of A. This indicates that the address allocated by peer A is allowed to receive packets from B.
- 4) *Switch Relaying:* Peer A will send a packet to the switch with an additional header that contains the public IP address and Port of B. The switch will intercept this packet and replace the destination address of the IP header with the address of peer B. It will then output this packet to the new address without passing through the relay server. Peer B will send a packet without an additional header to the $(relayIP, relayPort)$ address allocated by A. The switch will check its permissions table to see if the source IP and Port of B is allowed to send packets to A and, if allowed, relay the packet.

D. Building a switch network

Once the basic workflow for running relay sessions on the data plane is set, we can think about how to add multiple switches to the architecture. We consider multiple peers are distributed across a large geographical area. We want to place a set of programmable switches in strategic locations such that they can relay the connections with minimal latency. We detail the following challenges in such scenario:

Determining which switch is closest to the user: When relaying connections, we want the switch closest to the client to manage those connections. IP anycast will route incoming packets to the closest switch (as computed by the BGP protocol hops proximity metric) when advertising the same IP for all switches. Therefore, we split the architecture in two anycast domains: one for the set of switches and another for the controller. The set of switches will advertise their domain to the DNS server. Clients, when connecting to the switch anycast domain, will communicate with the closest switch.

Managing incoming packets: A peer A initiating a session on the relay server for the first time will connect to the data plane domain to allocate an IP. Upon receiving an allocation request, the switch will transmit the request to the controller with an ID indicating the origin of the request. Figure 4 shows the data plane processing for incoming packets, which will either be relayed or sent to the controller. The packets sent by A include additional TURN headers: MSG-TYPE, indicating if it is an *Allocation*, *Permission* or *Relay* packet; PEER-ADDR contains the IP and port the switch will use to replace the packet’s dst IP and dst UDP port; and an *UnknownAddr* bit to enable communication between peers relaying through different switches.

Updating the state of each switch: Upon receiving an allocation request, the controller will update the match-action tables only on the switch that is closest to the client, and will also update its own internal state. To manage multiple switches, the controller will maintain a copy of the state of every switch, to keep track of which connections are active at any point in time. A problem arises when peer A wants to connect with peer B in a distant region. The switch closest to peer B may not have peer A allocation in its internal state. The switch will not have enough information to determine what to

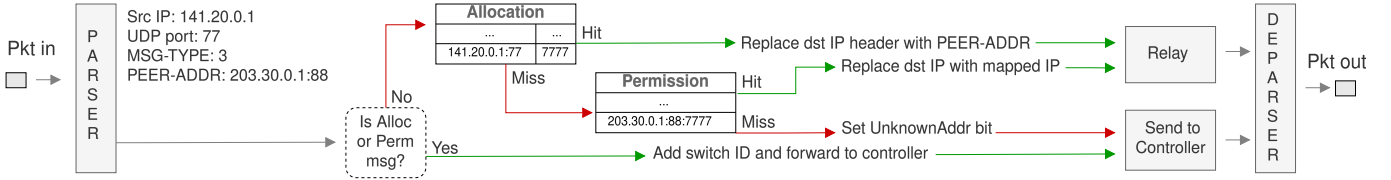


Fig. 4: P4-TURNet data plane path for incoming packets.

Algorithm 1: Managing switches on Controller

```

1  Switches: A dictionary mapping switch IDs to
2  their state data
3  init_network(Switches);
4  while listen() do
5    id ← extract_switch_id(pkt);
6    m_type ← extract_msg_type(pkt);
7    if m_type = ALLOC or m_type = PERM
8      then
9        ret ← update_tables(pkt, id);
10       add_entries(ret, Switches[id]);
11    else if m_type = UNKNOWN_DST then
12       res ← search_allocation(pkt, Switches);
13       if res ≠ NULL then
14         add_entries(res, Switches[id]);
15    else
16       /* Terminating connections */
17       delete_conn(pkt, Switches);
18       delete_entries(switches[id]);
19  clean_network(Switches);

```

do with the packet and will forward it to the controller. The controller, as shown in Algorithm 1, will then use its internal state to determine if an allocation exists and if peer *B*'s address is allowed to communicate with the allocation. If so, it will create both the allocation and the permission entries on the switch closest to *B*.

Managing the state in a distributed controller: If the controller is distributed, either to reduce the latency between the controller and the switch or to reduce load on the controller, it needs to propagate changes in its internal state to the other instances. This requirement does not impose a latency penalty on the solution, as communicating with the controller only happens during the presentation phase, which does not have strict latency or throughput requirements. Spreading a new allocation across all controller physical instances may take milliseconds up to a few seconds, whereas the relaying of packets in the second phase must happen at line rate. The main computation cost of the original TURN protocol is offloaded to the switches at the cost of implementing and maintaining an infrastructure of programmable switches. As the proposed relay server mostly deals with the presentation phase, it does not have strict computational requirements.

IV. EVALUATION

A. Setup

In our evaluation, we implement the relay server that acts as the controller in a virtual host and gather system usage server metrics. The system is evaluated in comparison to the traditional TURN technique of relaying packets on-server. We implement our system in a Mininet [8] emulation environment extended with the P4-Utils [9] project that provides tools to integrate programmable switch simulations into Mininet. The evaluation was performed on a QEMU x86-64 VM with 8 GB of RAM and one dedicated CPU core.

We compare a *Server Relay* architecture, in which all communication between two peers has to go through the server, with the architecture proposed in this paper (identified as *Switch Relay* in Figure 5). With Switch Relay, only the presentation and termination phases require communication with the server. We implement a 1-switch relay architecture and a 4-switch architecture to measure the additional server load required to control multiple switches. The clients are connected to the server via a virtual programmable switch and the number of simultaneous clients is simulated by increasing the traffic that goes into the switch. Due to the test environment's constraints, we are limited to simulate only four concurrent switches and a network load of up to 2 Mbps for each client. While a larger simulated network would be interesting, we focus on how the increase from 1 to 4 switches affects the load on the server relay.

B. Benchmarks

We evaluate each scenario with an increasing number of users until the limit of the simulated environment is reached (around 80,000 client hosts in our setup). Figure 5 illustrates the results for each measured metric (CPU, network and memory usage) on the server host.

CPU Usage: In Figure 5a we see that the Server Relay architecture reaches over 80% CPU utilization with around 3 thousand simultaneous users, a limit that is in line with the limitations reported by real-world TURN server implementations [6]. P4-TURNet reported a much lower CPU usage for both One-Switch and Four-Switch scenarios, due to all the computation required to relay packets being offloaded to the programmable switch. The resulting load that makes up the 20% usage is the handling of presentation and termination process and management of allocation and permissions table.

The overhead introduced from managing four switches versus a single switch in the P4-TURNet implementation is

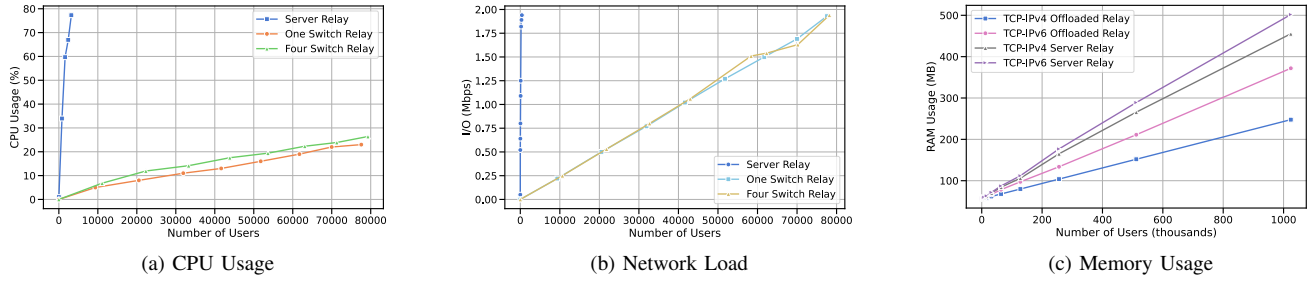


Fig. 5: Comparing the results of each scenario in a simulated environment

minimal, recording on average 2% more computational time required to process the additional communication. In Figure 6 we assess peer usage level: when each peer uses 2 Mbps of switch bandwidth, the server has an increased 10% of CPU usage compared to the lightest (124 Kbps) use. This slight increase is due to the number of additional control messages (such as `UnknownAddr`) that the server needs to process.

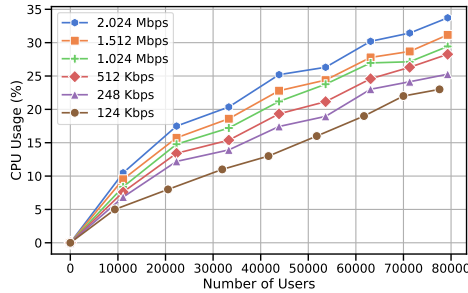


Fig. 6: CPU Usage with peer bandwidth variation.

Network Load: We measure the number of bytes received in the network interfaces connecting the server with the programmable switches in Figure 5b. Relaying packets on Server Relay quickly overloads the network interface, reaching full utilization with just 200 users. Due to the simulation environment constraints we are limited in stress testing the interface up to 2 Mbps. P4-TURNet is capable of handling 400 times more users before bottlenecking the network interface when it reaches around 80,000 clients. This bottleneck happened before the system overloaded the CPU or memory. However, this limit is unlikely to be an obstacle in a real life environment with a network interface capable of outputting Gigabits per second. The number of switches was not a relevant factor in network load, as network load is much more correlated with the number of simultaneous users connected to the system.

Memory Usage: We assess RAM usage in Figure 5c. We compare implementations with IPv6 addressing and IPv4 addressing. The number of switches connected to P4-TURNet does not impose an additional load on the server memory. IPv4 addressing records 35% lower memory consumption compared to IPv6. Server Relay records higher memory consumption in both IP versions due to requiring memory to relay packets in addition to managing the Allocation and Permissions table.

The results show that 500 MB of RAM are enough to store the table entries of over a million users for all implementations. Conversely, the trend in Figure 5a implies that a few hundred thousands users are sufficient to overload a CPU core. Therefore, memory does not constitute a bottleneck in the system, as network I/O and CPU capacities are exceeded much faster than memory is fully utilized.

C. Discussion

Our results indicate that a P4-TURNet server could handle hundreds of thousands of allocations given that the relaying phase is offloaded to programmable switches. A programmable switch can relay packets at extremely high speeds (commercially available models report throughput of over 10 Tbps) which is enough to guarantee a throughput of 10 Mbps for a million simultaneous users. However, the switch can be limited by the memory available for match-action tables. The least amount of data per connection is two tuples ($peerA_IP : peerA_Port, relayPort$) in the allocation table and its reverse lookup table; and ($relayPort, peerB_IP, peerB_Port$) in the permissions table. For IPv4, this sums up to 24 bytes per connection. Assuming a match-action table can support 10 MB of data, that means it can support 450k simultaneous connections. However, the maximum number of connections is also limited by the number of ports you can assign to the clients, which is 65,536 at most. Therefore, the solution supports at most 65k simultaneous users under CG-NATs.

To circumvent this constraint we could design a custom port header with more available bits to select from, such that the number of possible ports is more than the amount the switch memory supports. However, there is a subtle advantage in using TCP ports: in this solution, a peer B that is not under a CG-NAT does not need to have a custom client software to connect with A . In fact, it does not even need to know that A is using a relay server to communicate with B . All it needs is the address (the allocated address) of A . This transparency is intended by NAT traversal protocols such as TURN. If peer B is also under a CG-NAT, then both A and B will need to allocate addresses in the relay server. Another way to overcome this limitation is to have multiple IP anycast addresses supported by the system, for example one address for each interface in the programmable switch.

V. RELATED WORK

The study of programmable switches for peer-to-peer communication is still in its early stages. In [10] and [11], the authors use SDN to improve P2P video streaming over large networks. In [12], the authors optimize Distributed Hash Table overlays, a common architecture in P2P networks, with programmable switches to reduce maintenance traffic. In [13], [14], [15], [16] the authors employ SDN architectures to improve security and detect attacks on P2P networks. Offloading resource intensive applications to the switch has been explored recently in [17], [18].

Notably, in [7] the authors propose a solution to offload the relaying of packets from a relay server to a programmable switch by implementing the SIP signaling protocol to manage the state (i.e. the IP allocations and current connections) on the switch and use RTP as the media protocol for the relaying of packets. The experiments show that offloading the relaying of packets to the switch reduces the peak CPU consumption for the relay server to 15% during the presentation phase and less than 1% during the relaying phase. Additionally, their stress test environment for testing the maximum number of simultaneous connections reported maximum 400,000 media sessions for relaying with a switch and a maximum of 900 for relaying without a switch. P4-TURNet builds up on those results to extend the architecture across multiple switches.

Before the advent of programmable switches, previous work have studied IP anycast in P2P communication. In [19], the authors compare the implementations of a few basic strategies for selecting relay servers in an anycast environment. They note that the best latencies were achieved only when the relay servers were geographically close to the source or destination. In [20] the authors propose an anycast-based routing protocol for mobile ad-hoc networks, where there is no central server coordinating the devices. In [21], the authors explore the reverse path by implementing anycast on top of a P2P solution, Pastry, that computed its own locality information based on the delay value between active nodes in the system. Our contribution differs from these works by presenting an agnostic solution which can be integrated on any system that already supports TURN connections.

VI. CONCLUSION AND FUTURE WORK

In this work we proposed P4-TURNet, a system for enabling NAT traversal through a relay network using programmable switches for P2P applications with strict latency and throughput requirements. We discussed the basic workflow of address allocation and permission management for a peer under a CG-NAT. Then, we discussed how to scale the solution to a large geographical area such that each peer communicates with the switch that is closest to them. As future work, we consider adapting the solution to use a custom address for each allocation instead of IP:port as is usual in NAT traversal protocols. We also plan to extend P4-TURNet to support client mobility (i.e. client changing IP mid-session) and implement allocations with minimum bandwidth guarantees.

ACKNOWLEDGMENTS

This work was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001, CNPq (grant #311276/2021-0) and FAPESP (grant #2020/05152-7 and grant #2023/00764-2).

REFERENCES

- [1] Y. Wu, K. Zhang, and Y. Zhang, "Digital twin networks: A survey," *IEEE Internet of Things Journal*, vol. 8, no. 18, pp. 13 789–13 804, 2021.
- [2] S. Selvaraj and S. Sundaravaradhan, "Challenges and opportunities in iot healthcare systems: a systematic review," *SN Applied Sciences*, vol. 2, no. 1, p. 139, 2019.
- [3] P. Matthews, J. Rosenberg, and R. Mahy, "Traversal Using Relays around NAT (TURN): Relay Extensions to Session Traversal Utilities for NAT (STUN)," RFC 5766, Apr. 2010. [Online]. Available: <https://www.rfc-editor.org/info/rfc5766>
- [4] S. Blin and F. Naggar-Tremblay, "Establishing peer-to-peer connections with jami," <https://jami.net/establishing-peer-to-peer-connections-with-jami/>, 2019, (Accessed on 02/07/2024).
- [5] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, "P4: programming protocol-independent packet processors," *SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 3, p. 87–95, jul 2014.
- [6] "Github - coturn/coturn: coturn turn server project," <https://github.com/coturn/coturn>, (Accessed on 02/07/2024).
- [7] E. F. Kfoury, J. Crichigno, and E. Bou-Harb, "Offloading media traffic to programmable data plane switches," in *ICC 2020 - 2020 IEEE International Conference on Communications (ICC)*, 2020, pp. 1–7.
- [8] "Mininet: An instant virtual network on your laptop (or other pc) - mininet," <http://mininet.org/>, (Accessed on 06/26/2024).
- [9] "Welcome to p4-utils's documentation! — p4-utils 1.0 documentation," <https://nsg-ethz.github.io/p4-utils/index.html>, (Accessed on 06/26/2024).
- [10] R. Kharga, A. Rianto, and I.-S. Hwang, "P2p locality-aware live iptv over sdn based fiwi network," in *2021 30th Wireless and Optical Communications Conference (WOCC)*, 2021, pp. 222–224.
- [11] R. Farahani, E. Çetinkaya, C. Timmerer, M. Shojafar, M. Ghanbari, and H. Hellwagner, "Alive: A latency- and cost-aware hybrid p2p-cdn framework for live video streaming," *IEEE Transactions on Network and Service Management*, vol. 21, no. 2, pp. 1561–1580, 2024.
- [12] N. Shukla, D. Datta, M. Pandey, and S. Srivastava, "Towards software defined low maintenance structured peer-to-peer overlays," *Peer-to-Peer Networking and Applications*, vol. 14, no. 3, pp. 1242–1260, May 2021. [Online]. Available: <https://doi.org/10.1007/s12083-021-01112-7>
- [13] D. C. Fortune, S. S. Mathurin, and S. Kalita, "Http-based peer-to-peer botnet detection using a machine learning bagging classifier," in *2024 2nd International Conference on Disruptive Technologies (ICDT)*, 2024.
- [14] L. Feng, X. Ni, Z. Ling, and L. Wang, "Strong Anonymous Communication System Based on Segment Routing Over SDN," *The Computer Journal*, vol. 66, no. 12, pp. 3092–3106, 11 2022.
- [15] C. Medina-López, L. G. Casado, V. González-Ruiz, and Y. Qiao, "An sdn approach to detect targeted attacks in p2p fully connected overlays," *International Journal of Information Security*, vol. 20, 2021.
- [16] D. Attanayaka, P. Porambage, M. Liyanage, and M. Ylianttila, "Peer-to-peer federated learning based anomaly detection for open radio access networks," in *ICC 2023 - IEEE International Conference on Communications*, 2023, pp. 5464–5470.
- [17] C. H. Song, X. Z. Khooi, D. M. Divakaran, and M. C. Chan, "Re-visiting application offloads on programmable switches," in *2022 IFIP Networking Conference (IFIP Networking)*, 2022, pp. 1–9.
- [18] H. Tu, G. Zhao, H. Xu, and C. Qiao, "Programmable device deployment for efficient network function offloading," *Computer Networks*, vol. 239, p. 110163, 2024.
- [19] J. Zheng, K. Li, and Z. Wu, "Selection algorithms for anycast relay routing," in *IEEE International Conference on Performance, Computing, and Communications*, 2004, 2004, pp. 21–27.
- [20] R. Cheng, H. Jin, K. Shi, and B. Cheng, "An anycast-based p2p routing protocol for mobile ad hoc networks," in *2005 1st IEEE and IFIP International Conference in Central Asia on Internet*, 2005, pp. 5 pp.–.
- [21] R. Zhang and Y. C. Hu, "Anycast in locality-aware peer-to-peer overlay networks," in *Group Communications and Charges. Technology and Business Models*, B. Stiller, G. Carle, M. Karsten, and P. Reichl, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2003, pp. 34–46.